Academic Records Indexer

A high-performance Binary Search Tree architecture for educational data management

Deeptanshu Kumar | 2026166028 | M.Tech AI & DS
Advanced Data Structures and Algorithms - Mini Project
Faculty: Dr. M. Vijayalakshmi | Anna University, CEG Campus, Chennai

The problem: a dynamic, searchable, sorted set

- A department keeps CIA 1, CIA 2 and end-semester marks for every student
- Records arrive and change all semester: admissions, mark entry, corrections, withdrawals
- Two access patterns: pull one student's marksheet instantly, print the whole class in roll order
- Constraints: roll numbers must be unique, memory must be released, invalid input must not crash
- Arrays need O(N) per lookup or O(N) per insert; the project needs both to be fast

O(log N)
insert, search, update, delete (average)

O(N)
sorted marksheet by in-order traversal

943x
faster lookup than a linear scan at N = 1,000,000

0 leaks
every node freed on exit, duplicates rejected

Roll number
the primary key: unique, numeric, cheap to compare

Why a Binary Search Tree

Structure	Insert	Search	Delete	Sorted listing
Unsorted array	O(1)	O(N)	O(N)	O(N log N) sort first
Sorted array	O(N) shift	O(log N)	O(N) shift	O(N)
Linked list	O(1) / O(N) sorted	O(N)	O(N)	O(N) if kept sorted
Hash table	O(1) avg	O(1) avg	O(1) avg	O(N log N) sort first
BST (this project)	**O(log N) avg**	**O(log N) avg**	**O(log N) avg**	**O(N) in-order**

- Only the BST is logarithmic for all three updates and still keeps the data ordered
- Hash tables are faster for a single lookup but have no order, so a marksheet needs an extra sort
- The price: a BST degenerates if keys arrive sorted (slide 12); balanced trees fix this

Core data structure: the BST node

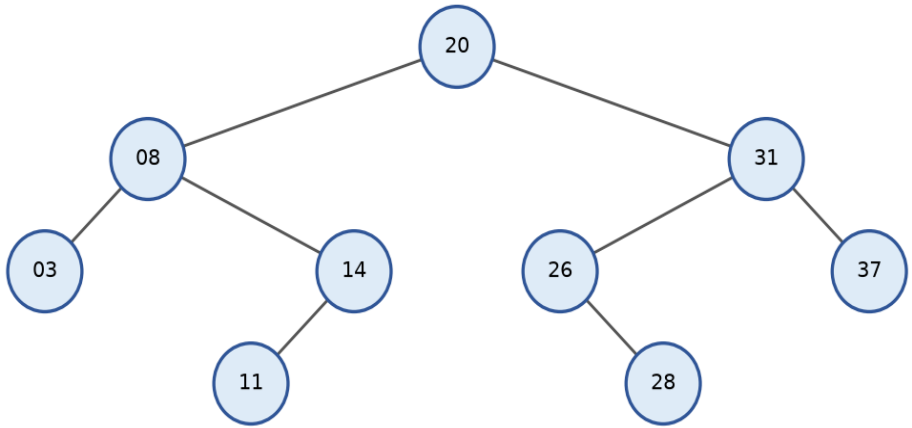
```
typedef struct StudentNode {
    long  roll_number;   /* primary key     */
    char  name[40];
    float cia1;          /* out of 50       */
    float cia2;          /* out of 50       */
    float end_sem;       /* out of 100      */
    int   is_eligible;   /* status flags    */
    int   has_arrears;
    struct StudentNode *left;   /* smaller */
    struct StudentNode *right;  /* larger  */
} StudentNode;
```

- Key: roll number, a long integer; one comparison per level
- Payload: three float marks, name for readable output
- Status: plain int flags, no <stdbool.h>; recomputed on every insert and update
- Links: left for smaller keys, right for larger; NULL at the leaves
- Allocated with malloc per record, freed in post-order on exit

sizeof(StudentNode) = 80 bytes on a 64-bit build; 1,000,000 records = 80 MB

Insertion: O(log N) average, duplicates rejected

BST after inserting 2026166020, 08, 31, 14, 26, 03, 37, 11, 28 (labels = last two digits)

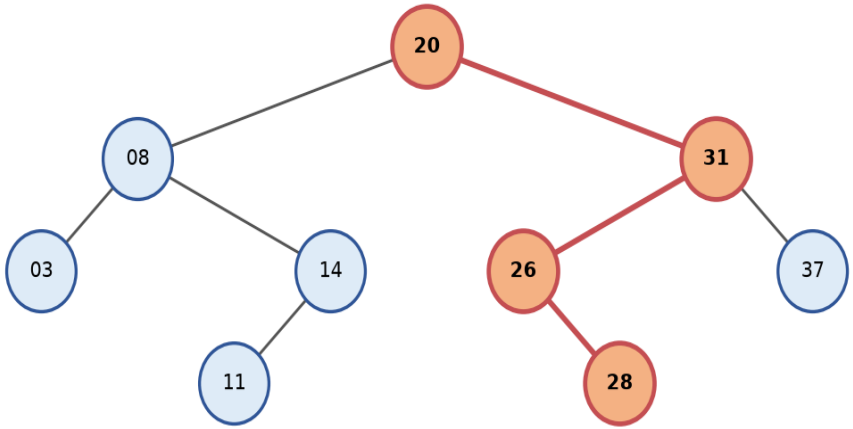


- Start at the root; smaller key goes left, larger goes right
- Walk until a NULL link is found and attach the new node there
- If an equal key is met, return 0: the roll number already exists
- Implemented iteratively with a pointer-to-pointer, no recursion
- Cost = number of levels walked = tree height h, about 2 log2 N for random order

The diagram shows the first nine records of students.csv (last two digits of the roll number).

Lookup: each comparison discards a sub-tree

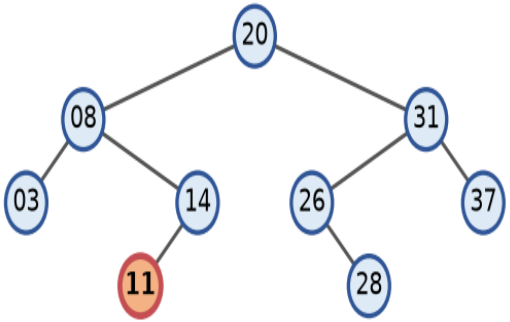
Search 2026166028: 20 -> right -> 31 -> left -> 26 -> right -> 28 (4 comparisons, 9 records)



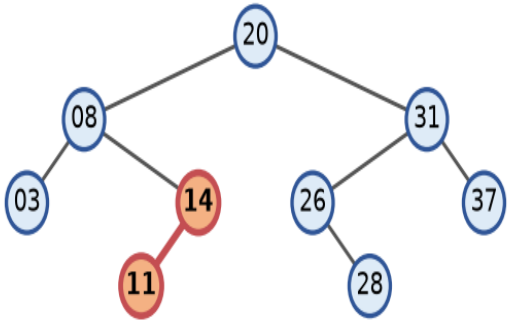
- Search 2026166028: 20 right, 31 left, 26 right, found: 4 comparisons
- Each step eliminates one whole sub-tree, roughly half the remaining records
- Unsuccessful search stops at a NULL link after at most h comparisons
- The program prints the comparison count after every search so the audience can verify it
- At N = 1,000,000 the benchmark measured 25.7 comparisons per lookup

Deletion: three cases, tree stays valid

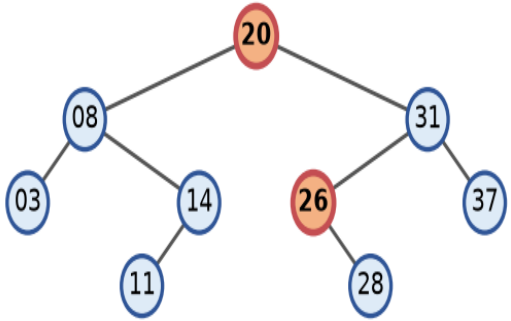
Case 1: leaf
delete 11: unlink and free



Case 2: one child
delete 14: splice child 11 into its place



Case 3: two children
delete root 20: copy successor 26, then delete 26



- Leaf: unlink from the parent and free
- One child: the parent adopts the child, then free
- Two children: copy the in-order successor (minimum of the right sub-tree) into the node, then delete the successor, which has at most one child

Marksheet by in-order traversal, O(N)

```
void inorder_print(const StudentNode *n)
{
    if (!n) return;
    inorder_print(n->left);   /* L */
    print_row(n);             /* Root */
    inorder_print(n->right);  /* R */
}
```

- Left, root, right visits keys in ascending order: the marksheet needs no sort
- Every node visited once with constant work: O(N) time, O(h) stack
- The same traversal writes the CSV export in sorted order
- Flags are stored, not recomputed per print, so the listing is a pure read

Quantity	Rule
Internal (40)	(CIA1 + CIA2) x 0.4
External (60)	end-sem x 0.6
is_eligible	internal percentage >= 40
has_arrears	total < 50, or not eligible
Grade	O >= 91, A+ >= 81, A >= 71, B+ >= 61, B >= 50, else U

Thresholds are #define constants at the top of the source and can be changed for another regulation.

Implementation: one C file, nine menu actions

1-4	**Insert, search, update, delete**	O(h) each; search reports comparisons; update recomputes flags
5-6	**Marksheet and statistics**	in-order listing; count, height vs balanced minimum, root, min, max, pre-order shape
7-8	**CSV load and save**	load any order, skip duplicates and malformed lines; save in-order
0	**Exit**	post-order free of every node, reports the count released
!	**Robust input**	scanf results checked, line flushed, marks range-checked, menu re-prompts

Benchmark method: clock() from <time.h>

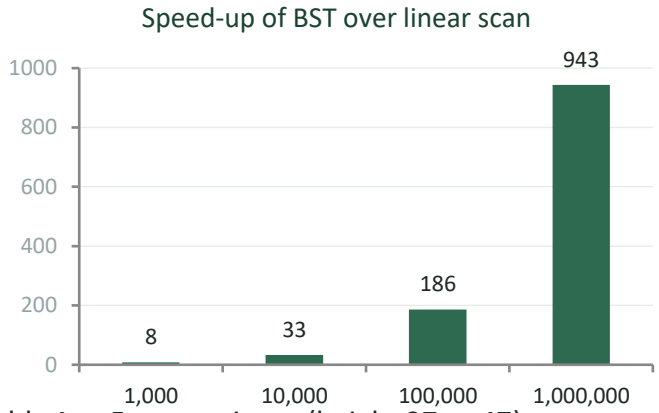
- Simulated datasets of N = 1,000, 10,000, 100,000 and 1,000,000 roll numbers, inserted in random order
- 1,000 random existing keys looked up by three methods: BST, linear scan of an unsorted array, binary search of a sorted array
- One lookup is faster than a clock tick, so each method repeats until 0.25 s of CPU time has elapsed and the average per lookup is reported
- Key comparisons are counted exactly: a machine-independent measure of the work
- Worst case reproduced separately: 10,000 keys inserted in sorted order

```
clock_t start = clock();
do {
    for (i = 0; i < 1000; i++)
        bst_search(root, keys[i]);
    total += 1000;
    elapsed = (double)(clock() -
start)
              / CLOCKS_PER_SEC;
} while (elapsed < 0.25);
avg_us = elapsed / total * 1e6;
```

Binary search on a sorted array is included as a fair O(log N) competitor; it cannot insert or delete in O(log N).

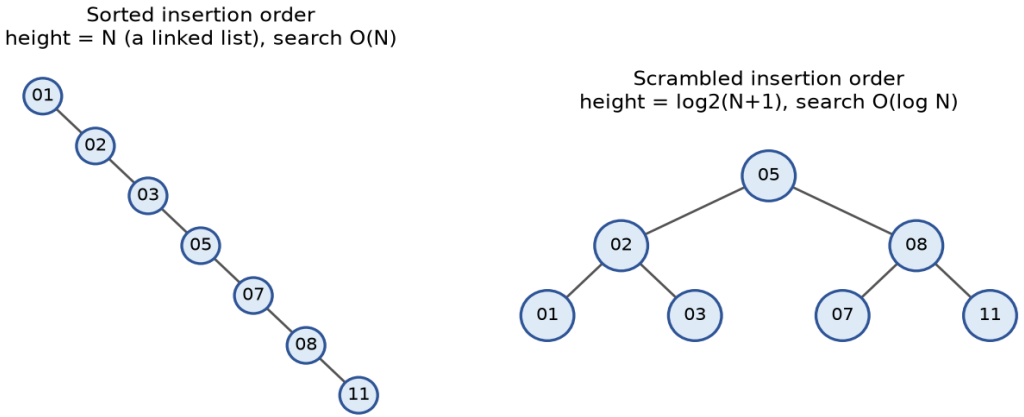
Results: log N against N

N	Height	BST cmp	Linear cmp	BST us	Linear us	Speed-up
1,000	27	11.4	487	0.015	0.113	8x
10,000	32	16.9	4,966	0.033	1.092	33x
100,000	41	21.2	49,060	0.058	10.792	186x
1,000,000	47	25.7	504,168	0.121	114.333	943x



- Ten times more records costs the linear scan ten times more work; the BST adds 4 to 5 comparisons (height 27 to 47)
- At N = 1,000,000: 25.7 comparisons and 0.12 us per lookup against 504,168 comparisons and 114 us
- Binary search on a sorted array matches the BST per lookup, but its inserts and deletes are O(N)

Worst case: sorted input degenerates the tree



- Measured: 10,000 keys inserted in ascending order give height 10000, 5053 comparisons and 15.8 us per lookup: O(N)
- Random order gives height about 2 log2 N; the sample CSV is deliberately scrambled
- Production fix: self-balancing AVL or Red-Black tree (rotations keep h <= 1.44 log2 N), or a B-tree on disk
- Reloading a sorted CSV would recreate the problem: insert the middle record first, recursively

Live demo: 13 steps on the sample class

6	Statistics: 25 records, height 5 = balanced minimum, root 2026166020
5	Marksheet in roll order with eligibility and arrear flags
2	Search 2026166028: found in 4 comparisons; search 2026166099: not found in 5
1	Insert duplicate 2026166028: rejected; insert new 2026166010: grade computed
3	Update 2026166026: arrears cleared, grade U to B
4	Delete leaf 2026166033, then delete the root 2026166020: successor 2026166023 moves up
6 5	Statistics and marksheet again: still height 5, still sorted
8 0	Save sorted CSV, exit: 24 nodes freed

Then: cd results && ..\bin\benchmark.exe (about 10 seconds)

Conclusion

- Delivered: O(log N) insert, search, update and delete; O(N) sorted marksheet; duplicate protection; computed eligibility flags; CSV persistence; leak-free shutdown
- Verified: 943x faster lookup than a linear scan at one million records, 26 comparisons instead of 504 thousand
- Limitation: a plain BST degenerates on sorted input; next step is an AVL or Red-Black tree, secondary indexes by name and department, and on-disk pages for a university-scale deployment

Thank you. Questions?